AWS CERTIFICATION EXAM PREP



CLF-C02

AWS Certified Cloud Practitioner

Complete Exam Notes

Domain-by-domain breakdown, exam-trigger tables, and a cheatsheet for every domain — condensed for exam week.

Notes by Mohammed Abdullah Khan · 3× AWS Certified

TABLE OF CONTENTS

AWS Certified Cloud Practitioner (CLF-C02) — Complete Notes

Domain 1: Cloud Concepts (24%)

- 1.1 What "Cloud Computing" Actually Means
- 1.2 Cloud Deployment Models
- 1.3 Cloud Migration Strategies (the "6 R's" — sometimes tested as 7)
- 1.4 AWS Well-Architected Framework — The 6 Pillars
- 1.5 Design Principles for Cloud Architecture
- 1.6 AWS Global Infrastructure
- 📋 Domain 1 Cheatsheet

Domain 2: Security and Compliance (30% — the single biggest domain)

- 2.1 The AWS Shared Responsibility Model
- 2.2 IAM (Identity and Access Management)
- 2.3 Compliance & Governance
- 2.4 Data Protection
- 2.5 Network Security
- 2.6 DDoS/Threat Detection Services
- 📋 Domain 2 Cheatsheet

Domain 3: Cloud Technology and Services (34% — the most points, the biggest service list)

- 3.1 Ways to Interact with AWS
- 3.2 Compute Services
- 3.3 Storage Services
- 3.4 Database Services
- 3.5 Networking & Content Delivery
- 3.6 Application Integration & Messaging
- 3.7 Management, Monitoring & Developer Tools
- 3.8 AI/ML & Analytics (lighter touch than the AI Practitioner exam, but still tested)
- 📋 Domain 3 Cheatsheet

Domain 4: Billing, Pricing, and Support (12%)

- 4.1 The Core Pricing Philosophy
- 4.2 Pricing & Cost Estimation Tools
- 4.3 Free Tier
- 4.4 AWS Support Plans
- 4.5 AWS Marketplace

4.6 Migration & Cost-Optimization Tools Worth Knowing Here

Domain 4 Cheatsheet

Master Quick-Fire Contrast Table (cross-domain last-minute review)

Traps That Cost Points at the High-Score Level

Self-Test — Explain These Out Loud (Feynman Check)

AWS Certified Cloud Practitioner (CLF-C02) — Complete Notes

Organized by the 4 official exam domains. Each domain ends with a **cheatsheet** — the compressed version to re-read the morning of the exam.

Domain	Weight
1. Cloud Concepts	24%
2. Security and Compliance	30%
3. Cloud Technology and Services	34%
4. Billing, Pricing, and Support	12%

Format: 65 scored questions (some unscored pretest), 90 minutes, multiple-choice/multiple-response. Passing score 700/1000. This is a **breadth-over-depth** exam — no hands-on labs, no CLI syntax, no architecture diagrams to draw. The bar is "could you explain this to a non-technical stakeholder and pick the obviously-right AWS service in a scenario."

Domain 1: Cloud Concepts (24%)

1.1 What "Cloud Computing" Actually Means

Cloud computing — on-demand delivery of IT resources (compute, storage, databases, networking) over the internet with **pay-as-you-go pricing**, instead of buying/maintaining physical infrastructure yourself.

The six advantages of cloud computing (AWS's official list — know these cold, they get tested directly):

1. **Trade capital expense for variable expense** — no big upfront data-center investment (CapEx); pay only for what you consume (OpEx).
2. **Benefit from massive economies of scale** — AWS aggregates usage across huge numbers of customers, driving per-unit costs down, passed on to you.
3. **Stop guessing capacity** — no more over-provisioning "just in case" or under-provisioning and running out; scale to actual demand.
4. **Increase speed and agility** — new resources available in minutes, not weeks of procurement.
5. **Stop spending money running and maintaining data centers** — focus on what differentiates your business, not racking servers.
6. **Go global in minutes** — deploy in multiple AWS Regions worldwide with a few clicks.

Exam trap: a question describing "no longer needing to predict server capacity months in advance" → **"stop guessing capacity."** A question about "shifting IT costs from upfront investment to ongoing operational spend" → **CapEx to OpEx.**

1.2 Cloud Deployment Models

Model	Description	Example
Cloud (public)	Fully in the cloud, no on-prem infrastructure	New startup running entirely on AWS
Hybrid	Mix of cloud + on-premises, connected together	Gradual migration, or data residency needs keeping some systems on-prem
On-premises ("private cloud")	Uses virtualization/resource pooling techniques on-prem, but not in a public cloud	Company running VMware internally for cloud-like flexibility, no AWS involved

Trigger phrase: "we want to migrate gradually and connect our data center to AWS" → **Hybrid** (services: **AWS Direct Connect**, **AWS Site-to-Site VPN**, **AWS Storage Gateway**, **Outposts**).

1.3 Cloud Migration Strategies (the "6 R's" — sometimes tested as 7)

Strategy	What it means
Rehost ("lift and shift")	Move as-is, no code changes — fastest
Replatform ("lift, tinker, and shift")	Small optimizations during move (e.g., switch to a managed DB) without changing core architecture
Repurchase ("drop and shop")	Replace with a different product, usually SaaS (e.g., move to Salesforce instead of self-hosted CRM)
Refactor / re-architect	Reimagine the app using cloud-native features — most effort, most long-term benefit
Retire	Turn off things no longer needed
Retain	Keep as-is (on-prem), e.g., not ready to migrate or has compliance constraints
Relocate (7th R)	Move VMs to cloud without changes, at hypervisor level (e.g., VMware Cloud on AWS)

Exam framing: "fastest way to migrate with minimal changes" → **Rehost**. "We want to leverage cloud-native scalability long-term and are willing to invest engineering time" → **Refactor**.

1.4 AWS Well-Architected Framework — The 6 Pillars

Pillar	Focus
Operational Excellence	Running/monitoring systems, improving processes/procedures
Security	Protecting data/systems, managing access (confidentiality, integrity)
Reliability	Recover from failures, meet demand, prevent/mitigate disruptions
Performance Efficiency	Using resources efficiently, keeping up with tech/demand changes
Cost Optimization	Avoiding unneeded costs, running at lowest price point that meets needs
Sustainability	Minimizing environmental impact of running workloads

Exam trap: a question emphasizing "using resources efficiently and adapting as demand changes" is **Performance Efficiency**, not Cost Optimization — Cost Optimization is specifically about the *price* dimension, Performance Efficiency is about *right-sizing/efficiency of the resource choice itself*.

1.5 Design Principles for Cloud Architecture

- **Scalability** — a system's ability to increase capacity to meet rising demand.
  - **Vertical scaling ("scale up")** — bigger instance (more CPU/RAM). Has a ceiling, usually requires downtime to resize.
  - **Horizontal scaling ("scale out")** — add more instances. No hard ceiling, matches cloud elasticity, works with load balancers.
- **Elasticity** — the system automatically grows *and shrinks* with demand (not just scales up) — this is the specifically "cloud" property, distinct from plain scalability.
- **High availability (HA)** — minimizing downtime, typically by running redundant resources across **multiple Availability Zones**.
- **Fault tolerance** — the system keeps working correctly even when a component fails (a stronger guarantee than just "available," implies no loss of function during the failure).
- **Disaster Recovery (DR)** — two metrics you must be able to define and pick strategies against:
  - **RTO (Recovery Time Objective)** — max acceptable *time* to restore service after a disruption ("how long can we be down").
  - **RPO (Recovery Point Objective)** — max acceptable *data loss* measured in time ("how much data can we afford to lose," i.e., time since last backup).
- **Loose coupling** — components communicate through well-defined interfaces (e.g., queues, APIs) so a failure/change in one doesn't cascade — enabled by services like **SQS**, **SNS**, **EventBridge**.
- **Microservices** — decomposing an application into small, independently deployable services — contrast with a **monolithic** architecture (one large tightly-coupled app).

1.6 AWS Global Infrastructure

Concept	Definition
Region	A physical geographic location containing multiple, isolated data centers (e.g., `us-east-1`). Choose a Region for latency, cost, and **compliance/data residency** reasons.
Availability Zone (AZ)	One or more discrete data centers within a Region, each with independent power/cooling/networking, connected via low-latency links. Every Region has ≥3 AZs. Deploy across multiple AZs for **high availability**.

Concept	Definition
Edge Location	Site used by **CloudFront** (and other edge services) to cache content close to end users, reducing latency — far more numerous than Regions/AZs.
Local Zones	Extend a Region closer to large population/industry centers for single-digit millisecond latency, for latency-sensitive apps (e.g., media production).
Wavelength Zones	AWS infrastructure embedded within telecom providers' 5G networks — ultra-low latency for mobile edge use cases.
Outposts	AWS hardware deployed **in your own data center**, for workloads needing to run on-prem while still using AWS APIs/services (hybrid).

Exam pattern: "How do I get high availability?" → deploy across **multiple AZs**. "How do I reduce latency for global users accessing static content?" → **CloudFront** (edge locations). "How do I keep data within a specific country for compliance?" → choose the right **Region**.

CHEAT SHEET

📋 Domain 1 Cheatsheet

- 6 advantages: **CapEx→OpEx, economies of scale, stop guessing capacity, speed/agility, stop running data centers, go global in minutes.**
- Deployment models: **Cloud** (all-in AWS), **Hybrid** (mix, connected via Direct Connect/VPN), **On-premises** (private virtualization, no public cloud).
- 6 R's: **Rehost** (lift-shift, fastest) → **Replatform** (tinker) → **Repurchase** (SaaS swap) → **Refactor** (re-architect, most effort/benefit) → **Retire** → **Retain**. (+**Relocate** = hypervisor-level move.)
- Well-Architected 6 pillars: **Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.** "Efficient resource use as demand changes" = Performance Efficiency, not Cost.
- **Scalability** (vertical=bigger box w/ ceiling; horizontal=more boxes, no ceiling) vs **Elasticity** (auto grows AND shrinks) vs **HA** (redundant across AZs, minimize downtime) vs **Fault tolerance** (keeps working despite failure, stronger than HA).
- **RTO** = how long can we be down. **RPO** = how much data can we afford to lose (time since last backup).
- **Loose coupling** via SQS/SNS/EventBridge lets components fail independently. Microservices vs monolithic.

- **Region** = geographic area w/ multiple AZs (pick for latency/cost/compliance). **AZ** = independent data center(s) within a Region (pick multiple for HA). **Edge Location** = CloudFront caching points (many more than Regions). **Local Zones** = low-latency extension near population centers. **Wavelength** = 5G telco edge. **Outposts** = AWS hardware on your premises.

Domain 2: Security and Compliance (30% — the single biggest domain)

2.1 The AWS Shared Responsibility Model

> ***AWS is responsible for security OF the cloud. The customer is responsible for security IN the cloud.***

AWS's responsibility ("of the cloud")	Customer's responsibility ("in the cloud")
Physical data center security	Data encryption choices & key management
Hardware/host infrastructure	IAM users, groups, roles, policies
Network infrastructure	Operating system patching (for EC2 — unmanaged)
Virtualization layer	Security group / firewall configuration
Global network, regions, AZs, edge locations	Data classification and handling
Managed service internals (e.g., RDS engine patching)	Client-side/server-side encryption, network traffic protection

The sliding scale rule: the more *managed* a service is, the more AWS takes on. For **EC2** (IaaS) — customer manages guest OS, patching, firewall config, IAM. For **RDS** (managed PaaS-like) — AWS patches the DB engine/OS, customer still manages IAM, data, network access, backups config. For **Lambda/S3** (fully managed) — AWS handles almost everything except your code/data/access policies.

Exam trap: "AWS handles it automatically" is **wrong** for IAM policy design, data encryption key choices, security group rules, and OS patching on unmanaged EC2 — those always stay the customer's job.

2.2 IAM (Identity and Access Management)

Core building blocks:

- **Root user** — created with the account, has unrestricted access. **Best practice: never use for daily tasks, enable MFA, lock away credentials.**
- **IAM Users** — individual identities for people/applications.
- **IAM Groups** — collections of users, for assigning permissions collectively.

- **IAM Roles** — a set of permissions **assumed temporarily** (by a user, an AWS service, or an external identity) — no long-term credentials attached. Trigger: "an EC2 instance needs to access S3" → **IAM Role** (never hardcode access keys on an instance).
- **IAM Policies** — JSON documents defining permissions (allow/deny on specific actions/resources).
- **Principle of least privilege** — grant only the minimum permissions needed — the "safe default" answer to almost any IAM question.
- **MFA (Multi-Factor Authentication)** — extra verification beyond password; strongly recommended on root and privileged users.
- **IAM Identity Center** (formerly AWS SSO) — centralized workforce access management across multiple AWS accounts + third-party apps, integrates with existing corporate directories.
- **AWS Directory Service** — managed Microsoft Active Directory in AWS, for integrating existing on-prem AD.

Exam trigger table:

Need	Answer
Temporary access for an AWS service to another AWS service	**IAM Role**
Someone in another AWS account needs access to your resources	**Cross-account IAM Role**
Centralize login across many AWS accounts	**IAM Identity Center**
Extra login security	**MFA**
Least-privilege access for a new employee	Custom **IAM Policy** attached to a **Group**

2.3 Compliance & Governance

- **AWS Artifact** — self-service portal for downloading AWS's own compliance reports/certifications (SOC, PCI-DSS, ISO, etc.) and reviewing/accepting agreements (e.g., BAA for HIPAA). **This is the answer whenever a question asks "where do I get proof AWS meets compliance standard X."**
- **AWS Config** — records and evaluates resource **configurations** over time against rules you define; answers "is this resource compliant with our configuration policy," and "what changed and when."
- **AWS CloudTrail** — logs **API calls / user activity** across your account (who did what, when, from where) — the audit trail for actions, not configuration state.
- **Amazon Macie** — uses ML to discover and protect **sensitive data (PII)** in S3.
- **AWS Trusted Advisor** — automated best-practice checks across 5 categories: **Cost Optimization, Performance, Security, Fault Tolerance, Service Limits.** Free tier = limited core checks; Business/Enterprise support = full check list.

Config vs CloudTrail — the #1 confused pair in this domain: - **Config** = "what does my resource look like / is it compliant" (configuration state over time). - **CloudTrail** = "who did what, when" (API

activity log).

2.4 Data Protection

- **Encryption at rest** — data encrypted while stored (e.g., S3 server-side encryption, EBS volume encryption) — managed via **AWS KMS (Key Management Service)**.
- **Encryption in transit** — data encrypted while moving over the network (TLS/SSL).
- **AWS KMS** — create/manage cryptographic keys used to encrypt data across AWS services. Two key types: **AWS-managed keys** (AWS handles rotation/lifecycle) vs **Customer-managed keys (CMK)** (you control policy, rotation).
- **AWS Secrets Manager** — securely store, rotate, and retrieve secrets (DB passwords, API keys) — includes automatic rotation.
- **AWS Certificate Manager (ACM)** — provision/manage free SSL/TLS certificates for AWS resources (e.g., load balancers, CloudFront).
- **Amazon Macie** (again, worth repeating here) — automated PII/sensitive-data discovery in S3.

2.5 Network Security

- **Security Groups** — **instance-level** virtual firewall attached to EC2/ENI. **Stateful** — if you allow inbound traffic, the response is automatically allowed out (no need for a matching outbound rule). Only supports **allow** rules (no explicit deny).
- **Network ACLs (NACLs)** — **subnet-level** firewall. **Stateless** — inbound and outbound rules must both be explicitly defined. Supports both **allow and deny** rules, evaluated in numbered order.
- **AWS WAF (Web Application Firewall)** — protects web apps from common exploits (SQL injection, XSS) at the **application layer (Layer 7)**, works with CloudFront/ALB/API Gateway.
- **AWS Shield** — DDoS protection. **Shield Standard** = automatic, free, protects all customers. **Shield Advanced** = paid, more sophisticated attack detection/mitigation, 24/7 DDoS response team access, cost protection.
- **AWS Firewall Manager** — centrally manage WAF rules and security groups **across multiple accounts** in an AWS Organization.
- **VPC (Virtual Private Cloud)** — your logically isolated network within AWS. Subnets (public/private), route tables, internet gateways (public internet access), **NAT gateways** (allow private subnet outbound internet access without allowing inbound).
- **VPC Peering** — direct network connection between two VPCs.
- **AWS Direct Connect** — dedicated, private network connection from on-prem to AWS (not over public internet) — for consistent bandwidth/lower latency than VPN.
- **AWS Site-to-Site VPN** — encrypted connection over the public internet between on-prem and AWS.
- **VPC Endpoints** — private connectivity from your VPC to supported AWS services **without traversing the public internet**.

Security Groups vs NACLs — memorize this contrast, it's tested directly:

	Security Group	NACL
Level	Instance	Subnet
Stateful?	Yes (return traffic auto-allowed)	No (must define both directions)
Rule types	Allow only	Allow **and** Deny
Evaluation	All rules evaluated	Evaluated in rule-number order, first match wins

2.6 DDoS/Threat Detection Services

- **Amazon GuardDuty** — intelligent **threat detection** using ML, analyzes CloudTrail/VPC Flow Logs/DNS logs to flag malicious/anomalous activity (e.g., compromised credentials, crypto-mining).
- **AWS Security Hub** — **centralized dashboard** aggregating security findings from GuardDuty, Macie, Inspector, Config, and third-party tools — a single pane of glass, checks against security standards (CIS benchmarks).
- **Amazon Inspector** — automated **vulnerability scanning** of EC2 instances, container images, and Lambda functions for known CVEs and unintended network exposure.
- **AWS Audit Manager** — continuously audit AWS usage to simplify assessing risk/compliance against a framework.

Quick trigger table for Domain 2's service zoo:

Concern	Service
"Is my resource configuration compliant / what changed"	**AWS Config**
"Who did what API call, when"	**CloudTrail**
"Find sensitive/PII data in my S3 buckets"	**Macie**
"Detect a compromised instance / malicious activity"	**GuardDuty**
"Scan EC2/containers for known vulnerabilities"	**Inspector**
"One dashboard for all my security findings"	**Security Hub**
"Protect my web app from SQL injection/XSS"	**WAF**
"Protect against DDoS"	**Shield**

Concern	Service
“Manage WAF/SG rules across many accounts”	**Firewall Manager**
“Download AWS’s compliance certificates”	**AWS Artifact**
“Store/rotate a database password”	**Secrets Manager**
“Manage encryption keys”	**KMS**
“Free SSL certificate for my load balancer”	**ACM**
“Best-practice check across cost/security/performance”	**Trusted Advisor**

CHEAT SHEET

📋 Domain 2 Cheatsheet

- Shared Responsibility: AWS = security **OF** the cloud (physical, host, network, managed-service internals). Customer = security **IN** the cloud (IAM, data encryption choices, OS patching on EC2, security groups, data classification).
- IAM: **Root** (lock away, MFA, don’t use daily) → **Users/Groups/Policies** (least privilege) → **Roles** (temporary, assumed — always the answer for “service X needs to access service Y”). **IAM Identity Center** = centralized multi-account/workforce login.
- **AWS Config** = resource configuration compliance over time. **CloudTrail** = API activity audit log (who/what/when). Don’t conflate.
- **AWS Artifact** = where to get AWS’s own compliance reports/certs.
- **Trusted Advisor** = automated checks across Cost, Performance, Security, Fault Tolerance, Service Limits.
- **KMS** = encryption key management. **Secrets Manager** = store/rotate secrets (DB creds, API keys). **ACM** = free SSL/TLS certs. **Macie** = ML-based PII discovery in S3.
- **Security Group** = instance-level, stateful, allow-only. **NACL** = subnet-level, stateless, allow+deny, ordered rules.
- **WAF** = app-layer (L7) protection vs SQLi/XSS. **Shield** = DDoS protection (Standard=free/auto, Advanced=paid/24-7 team). **Firewall Manager** = central WAF/SG management across accounts.
- **GuardDuty** = ML threat detection (analyzes logs for malicious activity). **Inspector** = vulnerability/CVE scanning. **Security Hub** = aggregated single-pane-of-glass dashboard for all findings.

- **Direct Connect** = dedicated private line to AWS. **Site-to-Site VPN** = encrypted tunnel over public internet. **VPC Endpoint** = private access to AWS services without public internet.

Domain 3: Cloud Technology and Services (34% — the most points, the biggest service list)

3.1 Ways to Interact with AWS

- **AWS Management Console** — browser-based GUI.
- **AWS CLI (Command Line Interface)** — scriptable command-line access.
- **AWS SDKs** — libraries for programmatic access in various languages (Python/boto3, Java, JS, etc.).
- **Infrastructure as Code (IaC):**
  - **AWS CloudFormation** — AWS-native, define infrastructure in JSON/YAML templates, provision/update/delete as a single "stack." Enables repeatable, version-controlled infrastructure.
  - **AWS Elastic Beanstalk** — PaaS-style: upload your code, Beanstalk handles provisioning (EC2, load balancer, auto scaling, monitoring) automatically — less control than CloudFormation, faster to get running.
  - **AWS CDK (Cloud Development Kit)** — define infrastructure using real programming languages, synthesizes to CloudFormation under the hood.

Beanstalk vs CloudFormation: Beanstalk = "just run my app, manage the infra for me" (opinionated, less control). CloudFormation = "let me precisely define every resource" (full control, more setup).

3.2 Compute Services

Service	Model	Trigger phrase
Amazon EC2	Virtual servers (IaaS) — full control over OS, most flexibility	"need control over the OS/instance," "custom server config"
AWS Lambda	**Serverless** functions — run code in response to events, no server management, pay per invocation/duration	"run code without managing servers," "event-driven," "intermittent workload"
Amazon ECS	Container orchestration (Docker), AWS-native	"run containers," simpler AWS-only orchestration
Amazon EKS	Managed **Kubernetes**	"need Kubernetes specifically," multi-cloud/portable container strategy

Service	Model	Trigger phrase
AWS Fargate	**Serverless compute for containers** — works with ECS or EKS, no EC2 instances to manage	"run containers without managing the underlying servers"
AWS Elastic Beanstalk	PaaS — upload code, AWS handles provisioning	"deploy a web app quickly without configuring infrastructure"
AWS Batch	Fully managed **batch processing** at any scale	"run hundreds of batch computing jobs"
AWS Outposts	AWS infra physically in your data center	"run AWS services on-premises"

EC2 purchasing options (frequently tested):

Option	Description	Best for
On-Demand	Pay by the hour/second, no commitment	Unpredictable workloads, short-term, testing
Reserved Instances (RI)	1 or 3-year commitment for a discount (up to ~75%)	Steady-state, predictable long-term usage
Savings Plans	Commit to a $/hour spend for 1 or 3 years, flexible across instance families/regions	Predictable spend with flexibility (more flexible than RIs)
Spot Instances	Bid on spare AWS capacity, steep discount (up to ~90%), can be **interrupted** with 2-minute warning	Fault-tolerant, flexible workloads (batch jobs, CI/CD, big data) — **never for critical/stateful workloads that can't tolerate interruption**
Dedicated Hosts	Physical server dedicated to you	Compliance/licensing requirements tied to physical hardware (e.g., BYOL)

Auto Scaling & Load Balancing (tie compute together): - **Amazon EC2 Auto Scaling** — automatically adds/removes EC2 instances based on demand (or a schedule) — the mechanism behind "elasticity" from Domain 1. - **Elastic Load Balancing (ELB)** — distributes incoming traffic across multiple targets (EC2 instances, containers, IPs) for availability and fault tolerance. Types: **Application Load Balancer** (L7, HTTP/HTTPS, content-based routing), **Network Load Balancer** (L4,

extreme performance/low latency, static IP), **Gateway Load Balancer** (for third-party virtual appliances).

3.3 Storage Services

Service	Type	Trigger phrase
Amazon S3	Object storage, virtually unlimited, durable (11 nines), accessed via API/HTTP	"store files/objects," "static website hosting," "backup/archive"
Amazon EBS	Block storage, attached to a **single** EC2 instance, like a virtual hard drive	"persistent storage for an EC2 instance," "boot volume"
Amazon EFS	**File storage**, NFS protocol, can be mounted by **multiple** EC2 instances simultaneously	"shared file system across many instances"
AWS Storage Gateway	Hybrid storage — connects on-prem apps to cloud storage (S3, Glacier) via NFS/SMB/iSCSI	"extend on-prem storage to the cloud," "hybrid backup"
AWS Snow Family (Snowcone/Snowball/Snowmobile)	Physical devices for **offline, large-scale data transfer** into/out of AWS	"petabytes of data, limited/no network bandwidth"

S3 Storage Classes (know the cost/retrieval tradeoff, this is heavily tested):

Class	Use case	Retrieval
S3 Standard	Frequently accessed data	Immediate
S3 Intelligent-Tiering	Unknown/changing access patterns — automatically moves objects between tiers	Immediate (auto-optimized)
S3 Standard-IA (Infrequent Access)	Accessed less often but needs fast access when needed	Immediate, but per-GB retrieval fee
S3 One Zone-IA	Same as Standard-IA but stored in **only one AZ** — cheaper, less durable/resilient	Immediate, cheaper, use for recreatable data
S3 Glacier Instant Retrieval	Archive, needs millisecond access	Milliseconds

Class	Use case	Retrieval
S3 Glacier Flexible Retrieval	Archive, access a few times a year	Minutes to hours
S3 Glacier Deep Archive	Lowest cost, long-term retention (7-10 years)	Hours (up to 12)

S3 features to know: Versioning (keep multiple versions of an object, protects against accidental delete/overwrite), **Lifecycle policies** (auto-transition objects between storage classes or expire them over time), **S3 Replication** (Cross-Region or Same-Region), bucket-level **encryption** and **access policies/Block Public Access**.

EBS vs EFS vs S3 — the classic contrast: - **EBS** = block storage, one EC2 instance at a time, like a hard drive. - **EFS** = file storage, many instances at once, like a shared network drive. - **S3** = object storage, not "mounted" to a server at all — accessed via API, unlimited scale.

3.4 Database Services

Service	Type	Trigger phrase
Amazon RDS	Managed **relational** DB (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, plus Aurora-compatible engines) — AWS handles patching/backups/failover	"managed relational database," "SQL," "don't want to manage DB admin tasks"
Amazon Aurora	AWS-built, MySQL/PostgreSQL-compatible relational DB — higher performance/availability than stock RDS engines	"high-performance relational DB, AWS-optimized"
Amazon DynamoDB	**NoSQL** key-value/document DB, serverless, single-digit millisecond latency at any scale	"NoSQL," "massive scale," "flexible schema," "key-value access pattern"
Amazon Redshift	**Data warehouse** — large-scale analytics/BI over structured data	"data warehousing," "run complex analytical queries across huge datasets"
Amazon ElastiCache	In-memory caching (Redis/Memcached)	"reduce database load with caching," "sub-millisecond latency"
Amazon DocumentDB	Managed **document** database, MongoDB-compatible	"MongoDB workloads"

Service	Type	Trigger phrase
Amazon Neptune	Managed **graph** database	"highly connected data," "relationships/graph queries," "social network / fraud graph"
AWS Database Migration Service (DMS)	Migrate databases into AWS with minimal downtime	"migrate an on-prem database to AWS"

Exam framing: relational/structured/needs SQL joins → **RDS/Aurora**. Massive scale, simple access pattern, flexible schema → **DynamoDB**. Analytics over huge historical datasets → **Redshift**. Need speed via caching in front of a DB → **ElastiCache**.

3.5 Networking & Content Delivery

- **Amazon VPC** — your isolated virtual network (see Domain 2 for security aspects).
- **Amazon Route 53** — managed **DNS** service; also does health checks and traffic routing policies (weighted, latency-based, geolocation, failover).
- **Amazon CloudFront** — **CDN** — caches content at edge locations for low-latency global delivery, integrates with S3/EC2/ALB as the origin.
- **AWS Global Accelerator** — improves global application performance/availability by routing traffic through the AWS global network (not caching, unlike CloudFront — it's for optimizing routing of dynamic/non-cacheable traffic).
- **AWS Direct Connect / Site-to-Site VPN** — hybrid connectivity (see Domain 2).

CloudFront vs Global Accelerator: CloudFront = **caches** content close to users (great for static/cacheable content). Global Accelerator = **routes** traffic more efficiently over AWS's backbone (great for non-cacheable, dynamic, or TCP/UDP traffic, e.g., gaming, VoIP).

3.6 Application Integration & Messaging

- **Amazon SQS (Simple Queue Service)** — fully managed **message queue**, decouples producers/consumers, messages persist until processed. **Standard queue** = at-least-once delivery, best-effort ordering. **FIFO queue** = strict ordering, exactly-once processing.
- **Amazon SNS (Simple Notification Service)** — **pub/sub** messaging, pushes messages to multiple subscribers (email, SMS, Lambda, SQS) simultaneously.
- **Amazon EventBridge** — serverless **event bus**, routes events between AWS services, SaaS apps, and your own apps based on rules.
- **AWS Step Functions** — orchestrate multiple Lambda functions / AWS services into visual **workflows** (state machines).

SQS vs SNS — the classic pair: SQS = queue, one message consumed by **one** consumer (pull-based). **SNS** = topic, one message delivered to **many** subscribers (push-based, fan-out).

3.7 Management, Monitoring & Developer Tools

- **Amazon CloudWatch** — metrics, logs, alarms, dashboards — the core **monitoring** service for AWS resources and applications.
- **AWS CloudTrail** — API activity audit logging (see Domain 2).
- **AWS Config** — resource configuration compliance tracking (see Domain 2).
- **AWS Systems Manager** — operational hub: patch management, run commands across fleets, parameter store for config/secrets, session manager (SSH-less shell access).
- **AWS Trusted Advisor** — best practice checks (see Domain 2).
- **AWS Organizations** — centrally manage/govern **multiple AWS accounts** — consolidated billing, **Service Control Policies (SCPs)** to restrict what accounts can do.
- **AWS Control Tower** — sets up a secure, multi-account environment (a "landing zone") based on best practices, builds on top of Organizations.
- **AWS CodeCommit / CodeBuild / CodeDeploy / CodePipeline** — the **CI/CD toolchain**: source control, build, deploy, and orchestrate the whole pipeline, respectively.
- **AWS Cost Explorer / AWS Budgets** — see Domain 4.

Organizations & SCPs: an **SCP** sets the maximum available permissions for accounts in an OU — it's a *guardrail*, not a grant (still need IAM policies within the account to actually allow actions). **Consolidated billing** = one bill across all member accounts, and volume pricing discounts apply in aggregate.

3.8 AI/ML & Analytics (lighter touch than the AI Practitioner exam, but still tested)

- **Amazon SageMaker** — build/train/deploy custom ML models (full lifecycle).
- **Amazon Rekognition** — image/video analysis.
- **Amazon Comprehend** — NLP (sentiment, entities).
- **Amazon Lex** — conversational chatbots.
- **Amazon Polly** — text-to-speech. **Amazon Transcribe** — speech-to-text.
- **Amazon Kendra** — enterprise search.
- **Amazon Bedrock** — access to foundation models (GenAI) via managed API.
- **Amazon QuickSight** — business intelligence/data visualization dashboards.
- **Amazon Athena** — serverless, run SQL queries directly against data in S3.
- **AWS Glue** — serverless **ETL** (extract, transform, load) service.
- **Amazon EMR** — managed **Hadoop/Spark** big-data processing framework.
- **Amazon Kinesis** — real-time streaming data ingestion/processing.

Exam framing for this list: these questions are usually just "match the one-line description to the service name" — depth is shallow, breadth is what's tested. Athena = "SQL on S3 without a database." Glue = "ETL." QuickSight = "dashboards/BI." Kinesis = "real-time streaming."

CHEAT SHEET

📋 Domain 3 Cheatsheet

- IaC: **CloudFormation** = precise, full control, JSON/YAML templates. **Elastic Beanstalk** = upload code, AWS provisions everything (less control, faster). **CDK** = IaC via real programming languages.
- Compute: **EC2** = VMs, full OS control. **Lambda** = serverless functions, event-driven, pay-per-invocation. **ECS/EKS** = container orchestration (AWS-native / Kubernetes). **Fargate** = serverless compute *for* containers (no EC2 to manage).
- EC2 pricing: **On-Demand** (flexible, no commitment) > **Savings Plans** (flexible $ commitment) ≈ **Reserved** (specific instance commitment) > **Spot** (cheapest, can be interrupted — never for critical stateful work) > **Dedicated Hosts** (compliance/licensing).
- **Auto Scaling** = add/remove instances on demand. **ELB** = distribute traffic (ALB=L7/content-based, NLB=L4/high-perf, GWLB=appliances).
- Storage: **S3** = object storage, API-based, unlimited. **EBS** = block storage, one instance at a time. **EFS** = file storage, many instances at once (NFS). **Storage Gateway** = hybrid on-prem↔cloud. **Snow Family** = offline bulk data transfer.
- S3 classes (cost ↓ / retrieval time ↑): **Standard → Intelligent-Tiering → Standard-IA → One Zone-IA → Glacier Instant → Glacier Flexible → Glacier Deep Archive.**
- Databases: **RDS/Aurora** = managed relational/SQL. **DynamoDB** = NoSQL, massive scale, low latency. **Redshift** = data warehouse/analytics. **ElastiCache** = in-memory caching. **DocumentDB** = MongoDB-compatible. **Neptune** = graph DB. **DMS** = database migration.
- **Route 53** = DNS + routing policies. **CloudFront** = CDN (caches at edge). **Global Accelerator** = routes non-cacheable/dynamic traffic over AWS backbone (not caching).
- **SQS** = queue, pull, one consumer per message. **SNS** = pub/sub, push, fan-out to many subscribers. **EventBridge** = serverless event bus/routing. **Step Functions** = visual workflow orchestration.
- **CloudWatch** = metrics/logs/alarms (monitoring). **Systems Manager** = patch/ops hub. **Organizations** = multi-account mgmt + consolidated billing + SCPs (guardrails, not grants). **Control Tower** = automated multi-account landing zone.
- CI/CD: **CodeCommit** (source) → **CodeBuild** (build) → **CodeDeploy** (deploy) → **CodePipeline** (orchestrate all of it).
- Analytics one-liners: **Athena**=SQL on S3. **Glue**=serverless ETL. **QuickSight**=BI dashboards. **EMR**=managed Hadoop/Spark. **Kinesis**=real-time streaming.

Domain 4: Billing, Pricing, and Support (12%)

4.1 The Core Pricing Philosophy

Pay-as-you-go for compute/storage/data transfer, with the "3 pricing fundamentals" AWS calls out: 1. **Pay for what you use** — no upfront commitment required for most services. 2. **Pay less when you reserve** — commit for 1-3 years (RIs, Savings Plans) for a discount. 3. **Pay less with volume-based discounts when you use more** — e.g., S3 pricing tiers get cheaper per-GB at higher usage.

Data transfer: inbound data transfer is generally **free**; **outbound** (to the internet) is generally **charged**, and data transfer **between Regions** is charged, while transfer **within the same Region/AZ** can be free or cheaper depending on service.

4.2 Pricing & Cost Estimation Tools

Tool	What it does
AWS Pricing Calculator	Estimate costs **before** deploying — model out a hypothetical architecture's monthly cost
AWS Cost Explorer	Visualize/analyze **historical** spend, forecast future costs, filter by service/tag/account
AWS Budgets	Set custom cost/usage budgets, get **alerts** when thresholds are exceeded (or forecast to be)
AWS Cost and Usage Report (CUR)	Most detailed, granular billing data available, delivered to S3 for deep analysis
AWS Billing Conductor	Customize/organize billing data for internal chargeback/cost allocation across business units

Exam trap: "estimate cost of a system I haven't built yet" → **Pricing Calculator**. "Understand why last month's bill was high" → **Cost Explorer**. "Get notified if we're about to overspend" → **Budgets**.

4.3 Free Tier

Three types of free offers: - **Always Free** — certain services free forever up to a limit (e.g., Lambda's first 1M requests/month). - **12 Months Free** — free up to specified usage limits, starting from account creation date (e.g., certain EC2/S3/RDS usage tiers). - **Trials** — short-term free trial starting when a specific service is first activated (varies per service).

4.4 AWS Support Plans

Plan	Who it's for	Key features
Basic	Everyone, free	Customer service & communities, documentation, Trusted Advisor (core checks only), Personal Health Dashboard
Developer	Individual dev/test, low-stakes workloads	Business hours email access to Cloud Support Associates, general guidance <24h response
Business	Production workloads	24/7 phone/chat/email, **<1 hour** response for urgent issues, full Trusted Advisor checks, **AWS Support API**, third-party software support (e.g., OS)
Enterprise On-Ramp	Growing orgs w/ production workloads	Business features + a pool of **Technical Account Managers (TAM)**, concierge support, <30 min response for critical
Enterprise	Business/mission-critical workloads	Dedicated **TAM**, **<15 minute** response for business-critical, proactive guidance, Concierge Support Team

Response times to memorize (approx.): - Developer: general guidance, business hours only. - Business/Enterprise On-Ramp/Enterprise: 24/7, escalating urgency → faster guaranteed response, topping out at **<15 minutes for critical/business-critical impact on Enterprise**.

TAM (Technical Account Manager) — a named/pooled technical contact, only included from **Enterprise On-Ramp and Enterprise** plans — proactive guidance specific to your architecture, not just reactive ticket support.

4.5 AWS Marketplace

Digital catalog of third-party software (AMIs, SaaS, data products) that runs on AWS — deployable directly into your account, billed through your existing AWS bill.

4.6 Migration & Cost-Optimization Tools Worth Knowing Here

- **AWS Migration Evaluator** (formerly TSO Logic) — analyzes your on-prem environment to build a data-driven business case/cost estimate for migrating to AWS.

- **AWS Application Discovery Service** — collects usage/config data from on-prem servers to plan migrations.
- **Reserved Instance / Savings Plans recommendations** (in Cost Explorer) — surfaces where committing would save money based on actual usage history.

CHEAT SHEET

📋 Domain 4 Cheatsheet

- 3 pricing fundamentals: **pay for what you use, pay less by reserving, pay less at volume.**
- Data transfer: **inbound free** (usually), **outbound charged**, **cross-region charged**.
- **Pricing Calculator** = estimate *before* building. **Cost Explorer** = analyze *historical* spend + forecast. **Budgets** = alerts on thresholds. **Cost and Usage Report** = most granular raw billing data. **Billing Conductor** = internal chargeback/cost allocation.
- Free Tier types: **Always Free** (forever, capped), **12 Months Free** (from account creation), **Trials** (from service activation).
- Support plans ladder: **Basic** (self-serve, free) → **Developer** (business-hours email) → **Business** (24/7, <1hr urgent, full Trusted Advisor, Support API) → **Enterprise On-Ramp** (+pooled TAM, <30min critical) → **Enterprise** (+dedicated TAM, <15min business-critical, Concierge).
- **TAM** only exists on Enterprise On-Ramp/Enterprise — proactive guidance, not just tickets.
- **AWS Marketplace** = third-party software, billed via your AWS bill.
- **Migration Evaluator** = build the cost/business case for migrating. **Application Discovery Service** = gather on-prem data to plan the migration.

Master Quick-Fire Contrast Table (cross-domain last-minute review)

Confusable Pair	Distinction
Scalability vs Elasticity	Scalability = can grow (up or out); Elasticity = automatically grows **and shrinks** with demand
High Availability vs Fault Tolerance	HA = minimize downtime via redundancy; Fault Tolerant = keeps functioning correctly *through* a failure (stronger)
RTO vs RPO	RTO = time to recover (downtime tolerance); RPO = data loss tolerance (time since last backup)
Vertical vs Horizontal scaling	Vertical = bigger instance, has a ceiling; Horizontal = more instances, no hard ceiling
Security Group vs NACL	SG = instance-level, stateful, allow-only; NACL = subnet-level, stateless, allow+deny, ordered
AWS Config vs CloudTrail	Config = resource configuration/compliance over time; CloudTrail = who-did-what API activity log
CloudFront vs Global Accelerator	CloudFront = caches content at edge; Global Accelerator = routes non-cacheable/dynamic traffic optimally
SQS vs SNS	SQS = pull queue, one consumer per message; SNS = push pub/sub, fan-out to many subscribers
EBS vs EFS vs S3	EBS = block, single instance; EFS = file, many instances (NFS); S3 = object, API-based, unlimited scale
RDS/Aurora vs DynamoDB	RDS/Aurora = relational/SQL, structured; DynamoDB = NoSQL, massive scale, flexible schema
Direct Connect vs Site-to-Site VPN	Direct Connect = dedicated private line (consistent bandwidth); VPN = encrypted tunnel over public internet (faster to set up)

Confusable Pair	Distinction
CapEx vs OpEx	CapEx = upfront capital investment (on-prem norm); OpEx = pay-as-you-go operational spend (cloud norm)
Reserved Instances vs Savings Plans	RIs = commit to specific instance family/region; Savings Plans = commit to $/hour spend, more flexible across instance types
Cost Explorer vs Pricing Calculator	Cost Explorer = analyze *actual past* spend; Pricing Calculator = estimate *hypothetical future* spend
IAM Role vs IAM User	Role = temporary, assumed, no long-term credentials (services/cross-account); User = persistent identity for a person/app
Trusted Advisor vs AWS Config	Trusted Advisor = best-practice checks across 5 categories (cost/security/performance/fault tolerance/limits); Config = tracks configuration compliance against *your own* rules
GuardDuty vs Inspector vs Security Hub	GuardDuty = ML threat detection from logs; Inspector = vulnerability/CVE scanning; Security Hub = aggregated dashboard of all findings

Traps That Cost Points at the High-Score Level

1. **"AWS handles it automatically"** — wrong for IAM design, encryption key choices, OS patching on EC2, security group rules — shared responsibility keeps these with the customer.
2. **Security Group vs NACL** — read for "instance" vs "subnet," and "stateful" vs "explicit both directions."
3. **Spot Instances** — never the right answer for a workload that "cannot tolerate interruption" or is stateful/critical, no matter how attractive the discount sounds.
4. **CloudFront vs Global Accelerator** — a question about *static/cacheable* content is CloudFront; *dynamic/non-cacheable* or gaming/VoIP traffic is Global Accelerator.
5. **Config vs CloudTrail** — "what does this resource look like / is it compliant" vs "who called what API."
6. **Support plan response times** — don't mix up Business (24/7, <1hr) with Enterprise (<15min business-critical) — and remember only Basic is free.
7. **RTO vs RPO** — RTO is about *time to restore*; RPO is about *how much data you can lose*. Don't swap them.
8. **"Fastest, cheapest way to get started"** in a migration context → **Rehost**, not Refactor (refactor is "most long-term benefit," not "fastest").

9. **SCPs are guardrails, not grants** — an SCP allowing an action doesn't grant it; IAM policies inside the account still must allow it too.
10. A vague "how do I reduce costs" question can point to several right-shaped answers (Reserved Instances, Savings Plans, Spot, right-sizing, S3 lifecycle policies) — match the specific keyword (steady-state vs interruptible vs storage aging) to the specific tool.

Self-Test — Explain These Out Loud (Feynman Check)

1. What's the difference between elasticity and scalability, in your own words?
2. Why is a Security Group "stateful" but a NACL is not — what does that actually mean for a return packet?
3. Give a scenario where Spot Instances are the right choice, and one where they'd be dangerous.
4. What's the difference between what AWS Config tracks and what CloudTrail tracks?
5. Under the shared responsibility model, name three things that stay the customer's job even on a fully managed service like RDS.
6. Why would you choose Global Accelerator over CloudFront for a real-time multiplayer game?
7. What's the difference between RTO and RPO, and how would each show up in a disaster recovery requirement?
8. When would you pick DynamoDB over RDS, and why?
9. What's the one-sentence difference between SQS and SNS?
10. Which AWS Support plan is the first to include a Technical Account Manager, and what does a TAM actually do that a regular support ticket doesn't?

About the Author

Mohammed Abdullah Khan

AWS Certified Cloud Practitioner

AWS Certified AI Practitioner

AWS Certified Solutions Architect Associate

I write concise, exam-focused AWS certification notes — distilled down to what's actually tested, organized by domain, with the exam traps called out explicitly. If these notes helped you pass, I'd love to hear about it.

W Portfolio
mohammedabdullahkhan.com

in LinkedIn
linkedin.com/in/mohammed-abdullah-khan-7b82a31a5

@ Email
mohammedabdullahkhan26523@gmail.com

Good luck on your exam.